

1 Manual regression checklist

Prepared 18 September 2026. This is a test plan, **not a record of passing tests**. A completed architecture todo or automated test does not establish that its native UI interaction works. Recent reported failures remain priority retests, particularly keyboard navigation, hover handoff, link opening and Explorer sizing. This document does not assert that their fixes are complete.

For each case, mark **P** (pass), **F** (fail), **B** (blocked), or **N/A** with a reason beside the box. Leave untested cases blank. Record failures by the stable case ID; do not silently skip a failing check or count an unavailable backend as a pass. Repeat intermittent checks at least ten times, not just until one succeeds.

1.1 Record this run

- Date / tester: _____
- Commit and uncommitted changes / executable path / build time: _____
- macOS version / hardware / display scale / external display: _____
- Typst, Tinymist and Poppler versions / selected and effective preview backend: _____
- Theme / UI scale / window size / autosave / miTeX / shortcut overrides: _____
- Fixtures used / baseline build for performance comparison: _____

1.2 Safe setup and useful fixtures

Use a disposable workspace outside your real project. Keep an untouched copy of every fixture. Use a disposable local Git repository for Git tests, with no production remote. Disable autosave while testing destructive edits; explicitly turn it on for the autosave cases. Write down settings before changing them and restore them afterward. Never kill all processes named Tinymist or alter installed tool binaries to simulate failure. Target only a positively identified test process; skip fault injection if its ownership is unclear.

Prepare these fixtures once:

- **Small:** `main.typ` with headings, a label/reference, inline and block math, a literal include of `chapter.typ`, an image, and a harmless HTTPS link. Add comments and strings containing dollars and backticks.
- **Sibling:** `other.typ` with unmistakably different text. Put some repeated words and Unicode text such as `café` `日本語` `α` `😊` in both files.
- **Index:** a valid computed include, such as the example below. Also keep a variant with a missing literal include for error testing.
- **Editing:** blank document; a document containing an existing fenced code block; long wrapped lines; nested headings/blocks; a copy with CRLF line endings and a copy without a final newline.
- **Assets:** two visually different images and PDFs, including a mixed-page-size or rotated PDF, and 1-, 20- and roughly 100-page PDFs. A PDF with links helps.
- **Scale:** a copy of a representative large real document and a workspace with many files. Avoid private content if you plan to share logs or recordings.
- **miTeX:** an empty Typst document, a compatible explicit MiTeX document, and an incompatible native-Typst-math document. Package availability is a prerequisite.
- **Git:** commit two text files, then create separated changes in one file, an untracked file, a rename and one independently staged change.

```
// Index fixture: valid compilation, not a literal-only dependency.  
#let chapter = "chapter.typ"  
#include chapter
```

Default shortcuts below use macOS Command and Option. If rebound, use the effective binding shown in Settings. Do not treat an intentionally disabled shortcut as a regression. Some older documentation describes removed behavior; the expected results here deliberately cover the newer user-facing contract.

1.3 Fast smoke route — do this first

Run B01–B04, E01–E03, H01–H07, T01–T04, S01–S03, V01–V03, W01–W03, and R01. This covers build identity, startup, the reported regressions, tab identity, durable saves, preview and multi-window routing. Then work through the remaining sections. Stop destructive testing if source text or saved bytes appear to cross documents; retain the fixtures for diagnosis.

2 Build identity, startup and window lifecycle

☐ B01 — Actually test the new executable

Do: Build with `cargo build --release`; wait for successful completion, then launch `./target/release/tiptoptyp /absolute/path/to/disposable/main.typ` directly. Record its hash with `shasum -a 256 target/release/tiptoptyp`.

Expect: The process uses that exact binary and opens the requested fixture. Do not launch by app name, Dock icon or bundle ID. An old logo/title layout is a reason to stop and verify identity, not to trust the test.

☐ B02 — Startup with saved layout

Do: Set Explorer to a comfortable width, quit normally and relaunch the same binary. Repeat with two document windows.

Expect: Explorer is usable immediately, not a tiny strip; saved sensible widths survive. No panic, invisible controls or startup focus loop.

☐ B03 — No document windows

Do: Close all document windows but leave the macOS app running. Inspect File/Edit, open Settings, then use File > New Window. Repeat using File > Open.

Expect: Both paths create a usable window without a graphics-context panic. Document-only New/Save/Copy actions are disabled when inapplicable; Settings still works.

☐ B04 — Empty workspace is not a closed window

Do: Close the last tab with its X or `Cmd+W`; then create or open a document.

Expect: The window and Explorer remain, with an empty-workspace view. New/Open work; `Cmd+Shift+W` or the window close button closes the window itself.

☐ B05 — New versus opened window appearance

Do: Compare a newly created window and one opened from an existing file; resize, minimize, restore and enter/leave fullscreen.

Expect: Both have correct shadow, rounded edges, traffic lights and reachable controls; no missing preview, stale rectangle or displaced click target.

☐ B06 — Display and scale transitions

Do: Move a window between available displays, change UI scale, and resize from wide to narrow and back. Repeat with Settings and a popup open.

Expect: Text remains legible; overlays stay associated with their owner; native preview bounds track the panel and do not cover the toolbar or adjacent panes.

3 Editor input, selection and focus

☐ E01 — Word and line navigation — priority retest

Do: In alpha beta gamma, place the caret in the middle. Try `Option+Left/Right`, `Cmd+Left/Right`, plain arrows, and Shift with each movement. Repeat after a preview-to-source jump, popup dismissal, tab switch and Settings close.

Expect: Option moves by words, Cmd by line boundaries, Shift extends selection, and plain arrows move the caret. `Cmd+A` working alone is not sufficient.

☐ E02 — Preview-to-source typing ownership

Do: Click rendered preview text to jump to source. Immediately type one character, move by a word, select a word and undo the edit.

Expect: The intended source range receives input, not the WebView or another window. The jump does not keep resetting the caret on subsequent frames.

☐ E03 — Selection and history

Do: Type, select by dragging, double-click a word, copy/paste, cut, undo and redo. Switch away and back between operations.

Expect: Exact text and selection are preserved; undo/redo belongs to that tab. No duplicate input or lost edit after a switch.

☐ E04 — Text fields do not edit the document

Do: In Find, Replace, Explorer search and a Settings text field, try `Cmd+A`, `Option+arrows`, copy/paste and undo.

Expect: Only the focused field changes. Hidden source text and other windows do not receive the shortcut; selection shortcuts do not trigger app commands.

☐ E05 — Unicode and input methods

Do: Edit around accented text, Greek, CJK and emoji. If available, compose text with an IME, confirm/cancel composition, then navigate and undo.

Expect: No split characters, shifted selection, lost composition or wrong diagnostic positions. One visible character is not confused with a UTF-8 byte offset.

☐ E06 — Editing transforms

Do: Toggle comments on one and several lines; indent/unindent; format valid source; undo each operation. Try formatting an invalid document too.

Expect: Transforms target the intended range and undo cleanly. A failed format leaves source intact with an understandable error, rather than partial replacement.

☐ E07 — Long lines, wrapping and caret visibility

Do: Navigate and select across long wrapped lines, blank lines and the final line. Toggle line wrapping, numbers and sticky context; resize.

Expect: Caret and selection stay aligned with glyphs and scroll into view. Gutter/sticky text does not overlap or intercept unrelated editor clicks.

4 Hover cards and popup interaction

☐ H01 — Repeated hover discovery — priority retest

Do: Hover successive symbols at the beginning, middle and end of a document, with no clicks. Leave and return; repeat on cold and warm content.

Expect: Eligible symbols consistently open after the configured delay. One symbol's popup does not suppress the next; whitespace does not show stale content.

☐ H02 — The reported adjacent-symbol example

Do: Hover every meaningful symbol in both lines below, including the subscript symbols. Move between them in both directions.

Expect: Available hover information is reachable consistently at each position; a missing card is not attributed to a particular symbol without investigation.

```
$  
v^L in C^oo([epsilon,t_*];H_sigma^k)  
v^L in C^oo([epsilon,t_*];epsilon^k)  
$
```

☐ H03 — Triangle handoff — priority retest

Do: From the symbol that opened a card, move slowly and quickly toward its top-left, middle and top-right edge. Repeat near the right/bottom window edges where placement changes.

Expect: The card survives the route toward it and remains usable when entered. It does not work only when moving down-left or along a lucky narrow path.

☐ H04 — Move-away dismissal

Do: After opening a card, move away from both its source and the card; also enter the card and then leave into unrelated content.

Expect: The popup dismisses, without leaving an invisible input-blocking window. Moving toward the card is not mistakenly classified as moving away.

☐ H05 — Editor scrolling with a card open

Do: Leave the pointer on a hovered symbol and scroll the editor with trackpad/wheel; repeat with the scrollbar and keyboard navigation.

Expect: The card dismisses and scrolling remains smooth. A stationary pointer does not immediately reopen it over newly scrolled text.

☐ H06 — Scrolling inside a long card

Do: Enter a long documentation card, scroll to the bottom and back, select/copy text, and use its keyboard-focus shortcut followed by Escape.

Expect: All content is accessible without a Read more button; the editor does not scroll behind it. Escape closes it and returns usable editor input.

☐ H07 — Browser links — priority retest

Do: Use a harmless link in a tooltip. Click once and observe response time/browser tab count. On a fresh card, try a quick double-click.

Expect: One activation opens one destination, dismisses the card promptly and does not freeze the editor. Repeated clicks during the same pending activation do not produce a delayed burst of browser tabs.

☐ H08 — Popup families and sizes

Do: Repeat enter/leave, Escape and scroll tests for diagnostics, settings hints, font samples, image/PDF hover and Git hunks. Compare short and long hints.

Expect: Behavior is consistent where interaction is supported. Short hints are compact; no giant empty card, useless repeated label, clipped content or stale image.

☐ **H09 — Owner closure and stale content**

Do: Open a hover, switch tab/document, close the owning tab, then open another popup. Repeat while loading an image or font sample.

Expect: The old card disappears and late results cannot resurrect it or change the new card. No orphan popup remains after its window closes.

5 Tabs, Explorer and project index

☐ **T01 — Stable tab identity**

Do: Open three different files; edit each differently, set distinct carets/scroll positions, switch repeatedly, and reopen a file already open.

Expect: Buffers, dirty flags, history and positions stay with their files. Reopening the same canonical path selects its existing tab rather than duplicating it.

☐ **T02 — Drag tabs, not the window**

Do: Drag the first tab past the last and back, using short and long drags. Repeat with fixed-width tabs, overflow and a dirty tab.

Expect: The order changes and the window stays still. Active/preview identities and close/eye actions follow the file, not its former position.

☐ **T03 — Active tab versus preview source**

Do: Open a Typst file first, then another Typst file. Select a different preview source using its eye; edit and switch between both.

Expect: Initial preview uses the first eligible source. Changing the eye changes preview ownership independently of editor selection; choosing it again is harmless.

☐ **T04 — Asset tab with pinned Typst preview**

Do: Keep a Typst preview selected; open a PDF then an image. Scroll/zoom the asset, then return to source.

Expect: The asset replaces the code pane, not the pinned output. Its controls do not navigate the other pane. Non-previewable tabs have no closed-eye icon.

☐ **T05 — Tab control geometry and overflow**

Do: Inspect selected/unselected tabs in both themes; toggle fixed tab width; open enough tabs to overflow and navigate by keyboard.

Expect: Highlight includes name and both controls; X and eye are vertically aligned. Active tabs reveal themselves; truncation does not hide every usable control.

☐ **T06 — Close and preview reassignment**

Do: Close an inactive tab, active tab and selected preview tab in separate trials; cancel a dirty-tab close.

Expect: Only the intended tab closes; a valid surviving preview choice is made or output clears. Cancel preserves text and tab order; no unrelated document disappears.

☐ **X01 — Explorer width survives use — priority retest**

Do: Resize Explorer, toggle Cmd+1 off/on ten times, switch tabs and modes, resize the window narrow/wide, then relaunch.

Expect: A useful chosen width is retained when space permits; temporary constraints do not permanently save a tiny width. Dragging remains possible.

☐ **X02 — Selection does not reshape the file list**

Do: Scroll a long tree, expand folders and select files at several depths.

Expect: Scroll and expansion are retained; the active filename may be heavier but is not larger. Selecting a file does not unexpectedly reset the tree.

☐ **X03 — Explorer sections and navigation**

Do: Resize/collapse sections; search files, contents, symbols and references where supported; select a heading, label and subfile.

Expect: Results belong to the current workspace/document and navigate to correct lines; section splitters do not collapse neighboring content unexpectedly.

☐ **X04 — Explain the exact unresolved dependency**

Do: Open the computed-include fixture and inspect the workspace-header details; repeat with a missing literal include.

Expect: Details identify file, line and expression/path, explain missing Explorer entries versus compilation, and say what to do. A bare dynamic-dependency count is a fail.

☐ **X05 — Index refresh and limited results**

Do: Edit a heading in an unsaved open included tab; change preview source; refresh Explorer. With a very large include graph, inspect completeness details.

Expect: Current open-buffer results replace old ones; no stale sibling headings appear. A traversal limit is disclosed, not presented as a complete index or compile error.

☐ **X06 — Filesystem changes and safe operations**

Do: In the disposable tree create, atomically replace, rename and remove a file externally. Try Explorer rename, file drop/copy and deletion; cancel first.

Expect: Tree updates without jumping to another root. Cancel changes nothing; accepted operations affect only their target and report conflicts/failures honestly.

☐ **X07 — Root switch while indexing**

Do: Open a large workspace, immediately change to a small unrelated root, then create a file in the old root. Try a symlink to a parent directory in the fixture.

Expect: Late results/events do not repopulate the old tree; no symlink recursion or endless scanning. Current files remain usable during indexing.

6 Saving, external changes and close safety

☐ **S01 — Save and verify actual bytes**

Do: Edit a saved file, save, inspect it independently, then close and reopen it. Repeat with Unicode and a path containing spaces.

Expect: Disk and reopened source contain exactly the intended text. A clean indicator appears only after success; unrelated files are unchanged.

☐ **S02 — Save As and rename ownership**

Do: Save As to a new path; edit/save again. Repeat for a preview-source tab and attempt a destination already open in another tab.

Expect: Only the new path receives later saves; title, index and services follow it. An already-open destination is protected rather than silently overwritten.

☐ **S03 — Dirty close decisions**

Do: With multiple dirty tabs, try tab close, window close and app quit. Exercise Cancel, Save and Discard in separate disposable trials.

Expect: Cancel preserves every buffer; Save closes only after durable success; Discard affects only approved documents. A later edit invalidates an earlier approval.

☐ **S04 — Autosave active and background tabs**

Do: Enable a noticeable autosave delay. Edit A, switch to B, edit B and wait. Inspect both files; repeat while Settings is focused.

Expect: Each eligible file saves its own latest text after its deadline. Parking a tab neither loses autosave nor saves another tab's buffer into it.

☐ **S05 — Edit during an outstanding save**

Do: On a disposable slow destination or large file, save and immediately make a distinctive second edit. Try another save and then close.

Expect: The older completion cannot mark the newer edit saved or approve its close. UI stays responsive. Mark blocked if no outstanding-save overlap was achieved.

☐ **S06 — Save failure and cancellation**

Do: Cancel Save As; separately try a known unwritable disposable destination. Restore permissions before leaving the test.

Expect: Text, dirty state and original identity survive. Errors identify the destination; no false Saved message or close after a failed write.

☐ **S07 — Clean external reload**

Do: With no local edits, change the file in another editor; repeat with atomic replacement, deletion and recreation.

Expect: Changed content reloads or the disappearance is explained; no stale clean buffer silently overwrites the external version on the next save.

☐ **S08 — Dirty external conflict**

Do: Make an unsaved edit, change the same file externally, then wait for autosave and try manual save. Repeat with the tab parked.

Expect: Both versions are protected by an explicit conflict path; autosave does not overwrite external work and the app does not silently discard local edits.

☐ **S09 — Two windows, one path**

Do: Open the same disposable file in two windows; edit/save different text in each, including nearly simultaneous saves.

Expect: No mixed bytes or cross-window clean-state updates. The second writer observes changed disk state rather than blindly overwriting the first.

☐ **S10 — Line endings and empty files**

Do: Open/save CRLF, no-final-newline, empty and Unicode fixtures without formatting; compare bytes, then make a small edit and undo.

Expect: No unexplained normalization, truncation or character corruption. Intentional formatting changes must be distinguished from ordinary save behavior.

7 Completion, automatic delimiters, search and folding

☐ **C01 — Typst and TeX context**

Do: Request completion in Typst code, math, strings, comments, a TeX fence and active miTeX dollars. Try TeX `\alpha` and other common commands.

Expect: Relevant candidates appear in supported contexts, including basic TeX symbols; unrelated language suggestions do not replace ordinary prose.

☐ **C02 — Completion edit and stale reply**

Do: Accept a candidate with keyboard and mouse, exercise snippet placeholders and undo; request another candidate then quickly edit or switch tabs.

Expect: Acceptance applies once at the intended range; one undo restores the prior text. Old responses cannot edit a different version/tab or reopen a stale popup.

☐ **C03 — Completion with Unicode and fonts**

Do: Accept after Unicode text and on a CRLF line; test reference suggestions and `text(font: ...)` search, scrolling and font samples.

Expect: Insertion ranges and caret are correct. Samples match fonts, stay bounded and do not stall ordinary typing or survive a closed picker.

☐ **C04 — Triple-backtick insertion — priority retest**

Do: Type an opening three-backtick fence with and without a language, then Enter. Repeat at EOF, mid-file and above a separate existing fenced block.

Expect: A matching closing three-backtick line is inserted and the caret is inside. A later existing opener is not mistaken for this new fence's closer.

☐ **C05 — Fence closing and undo**

Do: Repeat inside an existing fenced block, at its actual closing fence, and by pasting a complete fence; undo an automatically paired insertion.

Expect: No unwanted nested/duplicate closing fence, and undo does not leave stray delimiters. Existing source below the insertion remains unchanged.

☐ **C06 — Dollar spacing and block indentation**

Do: In ordinary Typst mode type a dollar then Space; separately type a dollar then Enter. Inspect whitespace and repeat with pairing disabled.

Expect: Space yields dollar-space-caret-space-dollar. Enter yields opening dollar, newline, two-space indent plus caret, newline, closing dollar—not a tab. Disabled pairing does not insert partners.

☐ **C07 — Other paired delimiters**

Do: Type parentheses, brackets, braces and quotes; type through a closer; backspace an empty pair; wrap selected text where supported. Repeat in strings/raw text.

Expect: No doubled closer, deletion of surrounding source or inappropriate insertion into literal text. Undo restores the expected edit.

☐ **F01 — Find visibility and toggling**

Do: Open/toggle Cmd+F, switch Find/Replace, resize, scroll and switch tabs.

Expect: Fields remain visible above the editor, not hidden but clickable. Closing Find releases its space/input capture and returns usable source focus.

☐ **F02 — Search modes and boundaries**

Do: Find repeated mixed-case/Unicode text; navigate forward/back through wraparound; enable case sensitivity and regex; enter an invalid regex and a zero-length match.

Expect: Highlights, counts and current match agree. Invalid regex is explained without freezing; zero-length matches do not loop indefinitely.

☐ **F03 — Replace and undo**

Do: Replace one, then all; undo both. Try replace shortcuts when Replace is hidden, then switch documents while search is open.

Expect: Only the intended document/query changes. Hidden replacement controls are exposed before a destructive replacement; undo recovers exact text.

☐ **F04 — Fold markers and source integrity**

Do: Collapse nested content, expand it, collapse all and expand all. Inspect the compact marker beside a Git hunk.

Expect: Collapsed line ends visibly indicate omitted content; markers are easy to hit. Folding changes neither saved text nor undo history.

☐ **F05 — Navigation into folds**

Do: Navigate to hidden text via Find, Problems, Explorer and preview source jump; use arrows across a collapsed region and toggle line numbers.

Expect: The destination is revealed with a visible caret; no selection in invisible rows, incorrect vertical jump or unusable leftover gutter.

8 Compilation, preview, diagnostics and recovery

☐ **V01 — Normal live compile**

Do: Edit the small document repeatedly; introduce and fix a syntax error.

Expect: Preview eventually shows the latest valid revision, never an older completion replacing a newer one. Error state is clear; usable prior output is retained.

☐ **V02 — Pause, compile and resume**

Do: Pause automatic updates, edit, use Compile explicitly, then resume. On an untitled document cancel and then accept the PDF destination dialog.

Expect: Pause prevents automatic updates; explicit Compile works and writes canonical output at the chosen destination. Cancel writes nothing; resume catches up.

☐ **V03 — Both navigation directions**

Do: Jump source to preview and preview to source in main and included files, after edits and with another tab active.

Expect: The intended document/range is selected, coordinates remain correct, and source keyboard input works immediately. Record stale-preview limitations separately.

☐ **V04 — Preview geometry under overlays**

Do: Switch Code/Split/Preview, resize splits, toggle Explorer/Problems, open Find, menus and popups, then cover/uncover the app.

Expect: Native preview follows its actual visible bounds and never draws or captures clicks over another panel, menu or hidden preview area.

☐ **V05 — Problems and diagnostic freshness**

Do: Create two different errors in different files, click Problems rows, fix them and quickly switch tabs while diagnostics arrive.

Expect: Rows and hover cards identify correct files/ranges; fixed and stale errors clear. No diagnostic from an old document is painted onto the current one.

☐ **V06 — Status and history**

Do: Trigger compile success, error, save, Git update and preview-ready messages; inspect status history.

Expect: The current message occupies the shared position left of the tick; no duplicate competing text. History retains useful details, and messages are understandable.

☐ **V07 — Tinymist failure policy — advanced**

Do: In an isolated test run, identify its Tinymist child PID, terminate only that child once, and observe recovery.

Testing five failures requires a controlled failing-server fixture; do not infer it from five unrelated successful restarts.

Expect: A transient failure waits then retries instead of immediately falling back. Five consecutive failures select fallback; duplicate notifications do not count twice. Record retry evidence or mark the repeated-failure portion blocked.

☐ **V08 — Recovery ownership**

Do: During a controlled restart, change preview source, pause/resume or close the window. Reopen and recover with a valid tool configuration.

Expect: Late readiness cannot resurrect a closed/wrong preview. At most one current service owns that window; status agrees with the effective backend.

☐ **V09 — Missing capabilities**

Do: Record tool preferences. Select an invalid custom tool path in a disposable test configuration, refresh toolchain status, then restore it.

Expect: The resolved fallback and reason are explicit. Editing, LSP, compilation, rasterization and link support are not all falsely marked available/unavailable because one tool is missing. Existing installed tools are untouched.

☐ **V10 — Canonical export**

Do: Export PDF after edits and while the preview uses dark-page rendering; open the exported file independently. Repeat after a failed compile.

Expect: Exported content/quality comes from canonical PDF bytes, not tinted raster pages. Failure does not masquerade as a new successful artifact or overwrite with junk.

9 PDF/image viewing and resource reuse

☐ **P01 — Raster fallback page coverage**

Do: Select the raster backend and open 1-, 20- and 100-page fixtures; scroll slowly, fling to the end and return to the beginning.

Expect: Every visited page eventually renders with correct spacing/order. Loading does not leave permanent blanks, duplicate pages or abrupt scroll-position resets.

☐ **P02 — Zoom and page navigation**

Do: Zoom/pinch around the pointer, fit width, navigate first/last pages and resize; repeat on rotated/mixed-size pages and an asset beside Typst output.

Expect: The right viewer changes; page geometry, pointer anchoring and bounds remain correct. Repeated zoom does not accumulate increasingly blurred copies.

☐ **P03 — Replacement during work**

Do: While a large PDF is loading, switch to a visibly different PDF; quickly change zoom/theme and switch back. Repeat during preview recompilation.

Expect: Old pages/themes cannot overwrite the new artifact. Valid previous output may remain during replacement, but is replaced consistently when new pages are ready.

☐ **P04 — Links and coordinate transforms**

Do: Click internal and harmless external PDF links after zoom, scroll and rotation. Repeat in an asset tab while another Typst preview is pinned.

Expect: Hotspots match visible links and affect the correct viewer; external activation is prompt and singular. Missing link-extraction support is disclosed.

☐ **P05 — Images and bad assets**

Do: Open transparent and large images; hover them in source and Explorer. Then open a corrupt image/PDF and a missing asset path.

Expect: Transparency/aspect ratio are correct; errors remain local and understandable. A failed asset does not clear unrelated Typst output or hang the editor.

10 miTeX projection and ordinary-mode isolation

☐ **M01 — Eligibility and auto-enable**

Do: Compare an empty compatible document, explicit MiTeX calls, native Typst math and a non-Typst file; test auto-enable on/off and manual off.

Expect: Only compatible documents offer activation. Auto-enable respects compatibility; unrelated edits/settings do not override manual off. Existing native equations are not silently reinterpreted as TeX.

☐ **M02 — Inline versus block canonical source**

Do: In active mode enter inline α , space-delimited α and a newline-delimited block. Save and inspect disk independently.

Expect: Inline becomes `mi`, block becomes `mitex`, with required imports added once. Displayed TeX stays readable; saved/service text is valid canonical Typst.

☐ **M03 — Unfinished math is safe**

Do: Leave a dollar expression incomplete; type, undo, try save/format, then complete it. Also try disabling mode with unrepresentable content.

Expect: The editor retains all work; invalid conversion cannot silently write different native math or partially apply a service edit. The reason is actionable.

☐ **M04 — Coordinate mapping**

Do: Put Unicode and multiple inline/block expressions before a hover, completion, error, heading, Git hunk and preview jump; exercise each feature.

Expect: Every range targets displayed text correctly despite generated imports/escapes. Edits apply once and undo within the current representation.

☐ **M05 — Round trip and mode transitions**

Do: Save, close/reopen, enable/disable and Save As a compatible fixture containing existing renderer calls. Compare saved source and rendered output.

Expect: No duplicate imports or accidental rewrites of unsupported explicit calls. Mode changes intentionally reset old-representation history, not source content.

☐ **M06 — Inactive-mode cost and subfiles**

Do: Compare normal editing with auto-enable off/on on an incompatible large file. Edit an included open buffer in interactive and CLI previews.

Expect: Ordinary documents do not acquire projection lag. Interactive preview uses synchronized open buffers; CLI fallback's known imported-files-from-disk limit is recorded rather than mistaken for successful unsaved synchronization.

11 Git status, hunks and protected actions

Only use the disposable repository. Verify results independently with `git diff` and `git diff --cached`; a UI success message is not enough. Do not push or pull a real remote as part of this checklist.

☐ **G01 — Status and compact hunk UI**

Do: Create added/modified/deleted lines and open each gutter hunk; narrow the popup.

Expect: Counts use colored plus/tilde/minus notation; hunk actions share a compact title row and remain reachable. Diff text matches the intended file and change.

☐ **G02 — Hunk navigation**

Do: Use Previous/Next buttons and effective shortcuts from a popup and from the caret, including at the first/last hunk and inside folded content.

Expect: Navigation wraps correctly and reveals the selected hunk without changing text. Stage/revert shortcuts do not accidentally navigate instead.

☐ **G03 — Stage and unstage exactly one hunk**

Do: Stage one of two separated hunks, including unsaved text; unstage it. Keep an unrelated staged change elsewhere and inspect both diffs after each action.

Expect: Only the selected index change moves. The working file is not implicitly saved, and unrelated staged work survives.

☐ **G04 — Revert is an editor edit**

Do: With autosave off, revert one hunk then undo; inspect disk and the Git index.

Expect: Only that buffer range reverts, one undo restores it, and disk/index are not directly changed. Re-enable autosave only after checking this distinction.

☐ **G05 — Concurrent and stale actions**

Do: Request a hunk operation and immediately use a panel action or another window on the same repo; change the baseline externally before a subsequent action.

Expect: Operations serialize or reject safely; stale patches do not broaden their target. Completion updates the owner, not whichever tab is now active.

☐ **G06 — Path and repository edge cases**

Do: Repeat hunk actions on new files, an unborn repo, Unicode/quoted paths, CRLF, no final newline and a deliberately conflicted disposable file.

Expect: Supported cases preserve exact content; conflicts/unsafe overlap reject with useful errors. No unrelated staged content is lost.

☐ **G07 — Repository panel and optional local remote**

Do: Review history, stage/unstage a file, cancel then make a disposable commit. If testing fetch/pull/push, use only a separately prepared local test remote.

Expect: The intended repo/branch is used; failures leave coherent status. Close during an accepted mutation does not falsely report cancellation or duplicate it.

12 Settings, themes, shortcuts and menu state

☐ **A01 — Compact responsive Settings**

Do: Resize Settings through wide, narrow and short sizes; search for autosave and toolchain controls.

Expect: Autosave label plus slider wrap as a group; status groups move intact to the next line. No horizontal waste, clipped controls or oversized trivial hints.

☐ **A02 — Theme combinations and adjustments**

Do: Choose light/dark/System mode; select a dark theme in the light slot and vice versa. Change invert, hue and other color adjustments, then reset.

Expect: Both sets are available, usual choices first. Controls are grouped sensibly, reset works, and obsolete explanatory clutter is absent.

☐ **A03 — Contrast and imported themes**

Do: Inspect code, selection, active file, diagnostics, Git colors, popups and disabled controls in both themes; import a valid then invalid theme file.

Expect: Foreground/background pairs remain usable. Import errors do not partially replace the theme, and preview-page treatment does not corrupt exported PDF.

☐ **A04 — Fonts and sample lifecycle**

Do: Change UI/editor fonts and available weights, enable ligatures, inspect CJK, open/scroll/close font pickers repeatedly.

Expect: Correct fonts and weights render without line/gutter drift, missing glyphs or lingering sample windows. Closing/reopening restores functional controls.

☐ **A05 — Effective shortcuts**

Do: Rebind one action, disable another, attempt a collision, then reset. Check Settings labels, native menus, title-bar menus and actual keys.

Expect: All use the effective binding consistently. Disabled/old bindings do not still fire; modifiers select the intended action once rather than multiple commands.

☐ **A06 — Toolbar/menu availability**

Do: Inspect File/Edit/View in dirty, clean, asset, empty-workspace and no-window states. Resize the title bar and toggle duplicate menus.

Expect: Availability matches across menus; document commands cannot affect hidden tabs. Order is traffic lights, ttt, menus, tabs, then right-aligned actions/panels.

☐ **A07 — Preferences and child-window close**

Do: Change a harmless shared setting, close/reopen Settings, restart the app, and compare two windows. Press Cmd+W while Settings or a popup owns focus.

Expect: Shared preferences persist and propagate without resetting editor state. Closing a child does not close an owner's tab; typing focus returns sensibly.

☐ **A08 — Packages and cancellation**

Do: Open/search the package manager; inspect installed/available versions, cancel an operation and a removal confirmation. Only remove a disposable test version.

Expect: Network/errors do not freeze editing. Removal targets exactly the confirmed version; no unrelated package or document changes.

13 Multiple windows and asynchronous ownership

☐ **W01 — Foreground interaction with background windows**

Do: Open three windows, two sharing a root and one using another. Type and scroll in one while another compiles a large document.

Expect: Input remains responsive and targets only the foreground editor. Background work does not repeatedly steal focus or repaint visible content incorrectly.

☐ **W02 — Shared data, local UI state**

Do: Give same-root windows different tree expansion/search, active tabs and preview sources; add a file externally, then close one window.

Expect: Both see the file update while retaining local selection/layout. Closing one does not stop the other's watcher or preview services.

☐ **W03 — Shortcuts across owners**

Do: Alternate focus between document windows, their previews, Settings and a popup; invoke Save, Find, Copy, Close tab and a navigation shortcut.

Expect: Only the correct owner handles each command, once. No root window steals a secondary window's input and no child command silently edits background text.

☐ **W04 — Late-result stress sequence**

Do: Request hover/completion/format/index/asset load in a large file, rapidly switch tabs, rename or Save As, then close the initiating tab. Repeat ten times.

Expect: Replies cannot change another document, clean a newer revision, show an old tooltip or repopulate closed UI. No crash or sustained busy state.

☐ **W05 — Settings after the last document closes**

Do: Keep Settings open, close all document windows, change a harmless preference, then create a new window and close/reopen Settings.

Expect: Settings remains usable without a visible editor owner; new documents receive the setting and no dormant/invisible document captures input.

14 Performance and resource regression checks

Use an optimized build, the same fixture, theme, scale and window size for each comparison. Let startup/first render finish before warm measurements. Do not run compilation of the Rust app, tests or another profiling job during measurements. Compare cold with cold and warm with warm; preserve the exact baseline executable identity if a baseline exists. These checks detect symptoms, not proof of GPU allocation bounds or thread-level causes. Use docs/performance.md for controlled profiling; Activity Monitor's Memory is not a direct GPU-memory measurement.

☐ **R01 — Idle baseline and Settings overhead**

Do: After warmup, observe a small document idle for 60 seconds in Activity Monitor; repeat with Settings open, then closed. Record CPU, memory and process count.

Expect: No sustained busy CPU or endless visible repaint at rest. Opening Settings should not introduce a large persistent idle load; record numbers, not just feel.

☐ **R02 — Popup interaction comparison**

Do: On the same large file, scroll for 20 seconds with no popup, then with a hover triggered, then inside a long card. Repeat each three times.

Expect: Editor scrolling dismisses the card and remains comparable. Cold expansion and warm scrolling are measured separately; no long stall or queued click burst.

☐ **R03 — Many tabs versus many windows**

Do: Compare 1 and 20 tabs in one window, then 1 and 3 windows, first idle and then typing/scrolling. Record child-process counts after settling.

Expect: Tabs do not spawn a service set each. Additional windows may own services but idle work and resource use do not grow without bound or freeze the foreground.

☐ **R04 — PDF residency and cycling**

Do: Record memory while visiting 1/20/100-page PDFs, repeatedly zooming and switching between two windows; close the assets and repeat the cycle three times.

Expect: Visited-page count does not imply permanent retention of every decoded page. Repeated cycles approach a plateau; pages evicted under pressure render again. Allocator/OS caches mean memory need not instantly return to the exact start.

☐ **R05 — Popup and font churn**

Do: Open/close long tooltips, font samples, Settings and asset hovers 30 times. Observe memory/process count after each ten, with short settling intervals.

Expect: No persistent new child window/process per cycle, steadily worsening latency, runaway memory trend or text corruption. First-use cache growth is distinguished from continued growth on identical repeats.

☐ **R06 — Workspace event storm**

Do: In the disposable large workspace create/rename a batch of files while editing; trigger several index refreshes and then stop.

Expect: Foreground input stays usable; the latest tree/index wins and activity settles. Repeated requests do not leave a long backlog of obsolete work.

☐ **R07 — Long-session soak and cleanup**

Do: For 30 minutes alternate edits, saves, tabs, previews, themes and windows. Then close test windows normally and inspect their previously recorded child PIDs.

Expect: No progressively slower scrolling, disappearing glyphs or stuck controls. Owned services exit after shutdown settles; unrelated user processes are untouched.

14.1 Measurement sheet

Workload / run	CPU / interval	Memory / children	Latency / notes
Idle, warm			
Settings open			
Scroll, no hover			
Hover cold / warm			
20 tabs / 3 windows			
100-page PDF / cycle			
After closing / settling			

15 Reporting failures and finishing a run

For each failure record: case ID; exact executable/hash and dirty revision; minimal fixture; starting focus/tab/preview owner; numbered actions; expected versus observed result; frequency (for example 3/10); whether cold/warm; elapsed stall time; console error/backtrace if present. A screen recording is particularly useful for flaky hover routes and focus transitions. Avoid sharing document contents, credentials or private paths unnecessarily.

If the app hangs, preserve the sequence and use the CPU-sampling procedure in docs/performance.md before restarting if feasible. If native layout is wrong, `TIPTOPTYP_UI_TRACE=1` can supply bounds evidence in a separate diagnostic run. Do not compare tracing-on timings to a tracing-off baseline. A framebuffer screenshot alone does not prove native WebView composition or keyboard delivery.

☐ Z01 — Restore test changes

Do: Restore recorded settings/tool paths; close disposable windows; inspect real project status without modifying it. Keep failing fixtures and evidence.

Expect: No personal documents, real repository index, production remote or tool binary was changed by testing. Any unavoidable test preference changes are documented.

☐ Z02 — Honest release decision

Do: List passed, failed, blocked and untested IDs. Rerun the smoke route after fixes, then the failing case and neighboring ownership/focus cases.

Expect: Data-loss, wrong-document mutation, panic, blocked normal navigation and severe interaction stalls remain release blockers. Untested or flaky is not passed; deferred features are not silently promoted to verified functionality.

Coverage follows the document/save/synchronization, preview/PDF, index/watch, completion/Git, child-view and capability boundaries of architecture todos 184–221, plus the reported regressions around 224–227. Deferred pop-out PDF, new TeX engines and experimental compatible binaries are not assumed implemented.